

APOSTILA

Introdução à Programação e Sintaxe Básica em C++

1) Ambiente de Desenvolvimento

- **Web (IDE Online):** <https://www.onlinegdb.com/> (Selecione a linguagem C++17 ou C++20)
- **Software:** Visual Studio Code (VS Code) + Compilador GCC/G++ (MinGW-w64)
- **Referências para instalação no VS Code no Windows:**
 - a) *Instalação do C/C++ no Windows e configuração no VS Code (Youtube)*
 - b) *Como configurar e compilar C e C++ no VS Code - Windows 11 (Youtube)*
 - c) *Arquivo com instruções no portal do aluno*

2) Sintaxe Básica em C++

```
#include <iostream>

using namespace std;

int main() {
    cout << "Meu primeiro código em C++!" << endl;
    return 0;
}
```

Detalhando os componentes:

- **#include <iostream>:** Importa a biblioteca padrão de entrada e saída do C++ (*Input/Output Stream*). Substitui a <stdio.h> utilizada em C.
- **using namespace std;:** Evita a necessidade de escrever std:: antes de comandos como cout, cin e endl.
- **int main() { ... return 0; }:** Ponto de entrada obrigatório em programas C++. Retornar 0 indica execução bem-sucedida ao sistema operacional.
- **cout << ...:** Operador de saída no console (*Console Output*).
- **endl:** Insere uma quebra de linha e limpa o buffer de saída (pode ser substituído por "\n").

3) Tipos Básicos / Primitivos em C++

O C++ herda os tipos básicos do C e adiciona o tipo booleano (bool) de forma nativa.

Tipo	Tamanho Típico	Descrição
bool	1 byte	Valores lógicos: true (verdadeiro) ou false (falso)

Tipo	Tamanho Típico	Descrição
char	1 byte	Um único caractere (letra, símbolo ou dígito)
int	4 bytes	Números inteiros
float	4 bytes	Números reais de ponto flutuante (precisão simples)
double	8 bytes	Números reais de dupla precisão (mais casas decimais)
void	0 bytes	Tipo sem valor ou ausência de tipo

4) Modificadores de Tipo (short, long, signed, unsigned) em C++

Os modificadores alteram o tamanho da memória alocada e a faixa de valores aceita por variáveis inteiras ou reais:

Modificador / Tipo	Tamanho Típico	Faixa de Valores (Aproximada)	Descrição
short int (ou short)	2 bytes	-32.768 a 32.767	Inteiro curto (economiza memória)
long int (ou long)	4 ou 8 bytes	Pelo menos 32 bits (2,14 $\times 10^9$)	Inteiro longo para valores maiores
long long int (ou long long)	8 bytes	-9×10^{18} a 9×10^{18}	Inteiro de grande capacidade
unsigned int	4 bytes	0 a 4.294.967.295	Apenas valores positivos (dobra o limite positivo)
unsigned short	2 bytes	0 a 65.535	Inteiro curto sem sinal
unsigned long long	8 bytes	0 a $1,8 \times 10^{19}$	Maior capacidade para números inteiros positivos
long double	8 a 16 bytes	Precisão estendida	Ponto flutuante com máxima precisão decimal

5) Estrutura Básica para Declaração e Inicialização de Variáveis

Para utilizar uma variável em C++, é necessário informá-la ao compilador descrevendo o **tipo** e o **nome**.

A) Sintaxe Padrão de Declaração:

```
tipo_da_variavel nome_da_variavel;
```

Exemplo: int quantidade; ou float preco;

B) Declaração de Múltiplas Variáveis em Uma Única Linha:

É possível criar mais de uma variável do mesmo tipo simultaneamente, separando os nomes por **vírgula**:

```
int a, b, c;
```

Exemplo: double largura, altura, profundidade;

C) Inicialização de Variáveis Logo na Declaração:

As variáveis podem ou não receber um valor inicial no momento em que são declaradas. Atribuir um valor inicial evita que a variável contenha "lixo de memória".

1. Atribuição Direta (Padrão C/C++):

```
float preco = 3.55;  
char opcao = 'B';  
int x = 10, y = 20, z = 30;
```

2. Inicialização Uniforme / Chaves (Exclusiva do C++11 em diante):

```
int idade{20};  
double saldo{1500.75};
```

3. Inferência de Tipo (auto em C++):

```
auto contador = 5;    // O compilador deduz que é do tipo int  
auto taxa = 1.99;    // O compilador deduz que é do tipo double
```

D) Regras Obrigatórias de Nomenclatura de Variáveis

:

1. **Sem espaços:** Não é permitido espaço no nome de uma variável. Use camelCase ou snake_case (Ex.: valorTotal ou valor_total).
2. **Não podem começar com números:** Devem começar com uma letra ou sublinhado _. (Ex.: 1numero é inválido; numero1 é válido).
3. **Sem caracteres especiais:** Não utilize acentuação, cedilha ou símbolos como @, #, \$, %.
4. **Sem palavras-chave reservadas:** Não use termos próprios da linguagem (int, float, return, class, if, while).
5. **Sensibilidade a Maiúsculas/Minúsculas (Case Sensitive):** valor, VALOR e Valor são tratadas como três variáveis completamente distintas.

6) Operadores e Tabela de Precedência

A tabela de operadores em C++ segue a estrutura de precedência e associatividade:

Categoria	Operadores	Associatividade	Exemplo
1. Parênteses e Acesso	() [] . -> ::	Esq → Dir	(a + b) * c
2. Unários	+ - ++ -- ! ~ (tipo) * & sizeof	Dir → Esq	-a, ++x, !flag
3. Multiplicativos	* / %	Esq → Dir	a * b / c
4. Aditivos	+ -	Esq → Dir	a + b - c
5. Deslocamento Bit a Bit	<< >>	Esq → Dir	x << 2
6. Relacionais	< <= > >=	Esq → Dir	if (a >= b)
7. Igualdade	== !=	Esq → Dir	if (a == b)
8. AND Bit a Bit	&	Esq → Dir	a & b
9. XOR Bit a Bit	^	Esq → Dir	a ^ b
10. OR Bit a Bit		Esq → Dir	a b
11. AND Lógico	&&	Esq → Dir	if (a > 0 && b > 0)
12. OR Lógico		Esq → Dir	if (a > 0 b > 0)
13. Condicional (Ternário)	?:	Dir → Esq	max = (a > b) ? a : b;
14. Atribuição	= += -= *= /= %= <<= >>= &= ^= =	Dir → Esq	x += 5;
15. Vírgula	,	Esq → Dir	x = 1, y = 2;

OBS: Na dúvida, sempre utilize parênteses em suas expressões para garantir a ordem de avaliação desejada.

7) Comentários

Todo trecho contido em um comentário é ignorado pelo compilador:

```
// Comentário de uma linha
```

```
/*
    Comentário de
    múltiplas linhas
*/
```

8) Comandos de Entrada e Saída (cin, cout e getline)

Em C++, os comandos de entrada e saída são baseados em **Fluxos de Dados (Streams)**, dispensando o uso de especificadores de formato (como %d e %f) e operadores de endereço (&).

A) Principais Funções e Operadores:

1. **cout <<**: Envia informações para a tela.
2. **cin >>**: Lê palavras únicas ou números. **Interrompe a leitura ao encontrar espaço em branco, tabulação ou quebra de linha (\n).**
3. **getline(cin, variavelString)**: Lê uma **linha inteira de texto**, incluindo espaços em branco, até que a tecla *Enter* seja pressionada.

B) Atenção ao Buffer do Teclado (cin.ignore()):

Ao ler um número ou palavra com `cin >>` e, em seguida, tentar ler uma linha inteira com `getline()`, a tecla *Enter* (\n) deixada no buffer de entrada fará com que o `getline()` pule a leitura. Para evitar isso, usa-se o comando **cin.ignore()** entre as leituras.

C) Exemplo Prático de Entrada e Saída Completa:

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    short idade;
    long long populacaoCidade;
    double salario;
    string nomeCompleto, cidade;

    // 1. Leitura de string com espaços
    cout << "Digite seu nome completo: ";
    getline(cin, nomeCompleto);

    // 2. Leitura de valores numericos usando short e long long
    cout << "Digite sua idade: ";
    cin >> idade;

    cout << "Digite a populacao estimada da sua cidade: ";
    cin >> populacaoCidade;

    cout << "Digite seu salario: ";
    cin >> salario;

    // Limpeza de buffer necessária antes do próximo getline()
    cin.ignore();

    // 3. Leitura de outra string com espaços
```

```

cout << "Digite sua cidade e estado: ";
getline(cin, cidade);

// Exibição dos dados
cout << "\n===== " << endl;
cout << "      DADOS CADASTRADOS      " << endl;
cout << "===== " << endl;
cout << "Nome: " << nomeCompleto << endl;
cout << "Idade (short): " << idade << " anos" << endl;
cout << "Populacao (long long): " << populacaoCidade << " habitantes" << endl;
cout << "Salario: R$ " << salario << endl;
cout << "Cidade: " << cidade << endl;

return 0;
}

```

9) Principais Diferenças entre C e C++ (Neste Estágio)

Recurso	Linguagem C	Linguagem C++
Biblioteca I/O	<stdio.h>	<iostream>
Saída de Dados	printf("Valor: %d", x);	cout << "Valor: " << x;
Entrada Simples	scanf("%d", &x);	cin >> x;
Entrada de Frase/Linha	fgets(str, tamanho, stdin);	getline(cin, str);
Tipo Booleano	Exigia <stdbool.h> (C99)	bool nativo (true/false)
Tratamento de Texto	Vetores de char (char str[50])	Tipo nativo std::string
Local de Declaração	Antigamente apenas no início do bloco	Em qualquer ponto do código
Organização	Sem suporte a namespaces	Utiliza namespace (ex.: using namespace std;)

Exercícios Práticos de Fixação

Exercício 1: Declarações e Nomenclatura

Analise as declarações abaixo e identifique quais são **válidas** e quais são **inválidas** segundo as regras do C++. Para as inválidas, explique o erro:

- a) `int 1valor = 10;`
- b) `double preco_produto = 45.90;`
- c) `float salario liquido = 2500.00;`
- d) `char opcao = 'A', status = 'F';`
- e) `int return = 0;`

Exercício 2: Cadastro com Múltiplas Variáveis e getline

Crie um programa em C++ que declare na mesma linha três variáveis inteiras do tipo short (dia, mes, ano). O programa deve solicitar:

1. O nome completo do aluno (usando `getline`).
2. A data de nascimento (lendo dia, mes e ano separados por espaço).
3. O nome do curso (usando `getline`). *(Não se esqueça de usar `cin.ignore()` antes do segundo `getline`).*

Exercício 3: Comparativo de Tamanhos em Memória (sizeof)

Escreva um programa em C++ que declare e imprima na tela o tamanho ocupado em bytes (`sizeof`) para cada um dos seguintes tipos: short, int, long, long long, float e double.

Exercício 4: Conversão de Tempo (Operadores Aritméticos)

Escreva um programa que solicite ao usuário uma quantidade inteira de segundos (long) e converta esse valor para **Horas, Minutos e Segundos**.

(Dica: utilize os operadores de divisão / e resto da divisão %).

Exercício 5: Avaliador de Expressões e Precedência

Sem executar no computador, avalie o resultado das variáveis booleanas e inteiras. Depois, programe em C++ para verificar seus resultados:

```
int a = 10, b = 4, c = 2;
bool r1 = (a + b * c) > 15;
int r2 = a % b + c * 3;
bool r3 = !(a > b) || (c < b);
```

Exercício 6: Conversor de Temperatura

Escreva um programa em C++ que leia uma temperatura em graus Celsius (double) e a converta para Fahrenheit e Kelvin utilizando as fórmulas:

- $F = (C * 9 / 5) + 32$
- $K = C + 273.15$